Relations Update Flow: Copilot Build Spec

Oct 12, 2026 · @Sn

1. Context, scope and constraints

Build a **new component and screen for updating relations** of a party, for both individual and organization party types. It handles **has-a joinings only**: the party being edited owns the participants (for example UBO and representative manager of a PLC, or parents and legal representatives of a minor individual). Each joining type allows at most 99 participants, so every section on this screen is bounded. Paste this section ahead of either the FE or the BE part when prompting.

Scope

- In scope: add and remove has-a joinings for the party, staged in the browser and submitted once as a delta. Works for individual and organization party types; the applicable joining types come from the joining metadata for the party type and legal form.
- Out of scope: is-a joinings (where the party is the participant in other parties' joinings, such as care taker companies or guardians of many parties). They are not shown or edited on this screen. No large-set handling, cursor stack or search-by-candidate scan is needed.
- Unchanged: the existing view relations screen and the existing all-joinings endpoint behind it. Do not modify them. The new screen is separate and uses its own components and endpoints, built on the same underlying APIs.
- Unchanged: the "reason no UBO" field.

Stack and target

- FE: Vue 3, TypeScript, Vite, Pinia, Alova. BFF: Quarkus, with the existing graph/process orchestrator and JTA writes.
- Desktop-only Windows intranet app (Chrome or Edge). Design for widths of 1280 px and up. No touch or mobile concerns.
- Follow existing project conventions for naming, folder layout, error handling and tests. If this spec conflicts with existing code conventions, follow the code and flag the conflict.

Config-driven rendering (must follow)

- The FE already has a meta-model config that describes screens and components, rendered by the common orchestrator shared component. This screen and every component added for it must follow that philosophy. Do not hand-wire the screen.
- Register `RelationsUpdateSection` and `LinkPartyModal` as component types in the shared orchestrator's component registry, and describe the screen in a meta-model config entry: which sections, their labels, columns, the component type per section, and the Link modal's tabs and search fields. Do not hardcode these per party type or legal form.
- Split of responsibilities: UI rendering choices come from the meta-model config. Business rules (applicable joining types for a party type and legal form, min, max, allowed participant kinds) come from the joining metadata API and are never copied into the config.
- Individual and organization variants differ only by config entries and metadata, not by separate screens or components.
- Before writing code, read the existing meta-model config schema and the shared orchestrator component, and mirror their conventions for naming, registration, props, events and how a store hooks in. If the illustrative config below does not fit the real schema, the real schema wins.

Illustrative config entry (keys and values are placeholders, adapt to the real schema):

```
{
  "screen": "updateRelations",
  "partyTypes": ["ORGANIZATION", "INDIVIDUAL"],
  "sections": [
    {
      "joiningType": "UBO",
      "component": "relationsUpdateSection",
      "label": "ultimateBeneficiaryOwners",
      "columns": ["name", "bcNumber", "partyId", "status"],
      "linkModal": {
        "component": "linkPartyModal",
        "title": "linkUltimateBeneficiaryOwner",
        "tabs": ["individualName"]
      }
    }
  ]
}
```

Terms

- **Joining**: one relation record between the main party and a participant, identified by `joiningId`.

- **Has-a**: the main party owns participants. At most 99 per joining type, enforced for all parties.
- **Section**: one joining type on the screen, for example Ultimate Beneficiary Owner(s).
- **Baseline**: the joinings as loaded from the server. **Delta**: the user's staged adds and removes on top of it.

Existing building blocks to reuse (do not re-implement)

- API that returns joinings of one joining type for a party. It returns BC numbers only, 15 rows per page, with keyset paging: the next page is requested with the createdTime and joiningId of the last record. It is strictly sequential and has no total.
- Joining metadata API: joining types, applicable party types and legal forms, min and max participants per type, allowed participant kinds (natural person or legal entity).
- Party search API by BC number, returning party ID and name (assumed; confirm).
- Upstream party details API that takes a batch of party IDs and returns date of birth, address and party ID.
- Existing party search screen and Link modal fields: Initials, Prefix, Surname, Date of birth, Country, Postcode, House number, with Search, Clear Search and Register. Search results are assumed to include the party kind (natural person or legal entity); confirm.
- Existing Verify screen (dual control) and the existing orchestrator endpoints used by the FE.

Non-negotiable rules

- No BFF-side cache and no new persistence. Load protection comes from lazy loading, batching, bounded concurrency and timeouts only.
- Edits are staged in the browser as a delta. Nothing is written until the user presses Next and the orchestrator accepts the payload.
- Submit sends only changed joinings: added and removed. Existing unchanged joinings are never resent.
- Concurrency between users is handled by the process orchestrator. No optimistic locking is added in the FE.
- Min and max rules come from the joining metadata API (for example a PLC needs at least one representative manager; max is 99). Do not hardcode them in the FE.

2. FE screen anatomy and styling

The screen is one page of the update wizard (step 1 of 4). The same page serves individual and organization parties: only the title subtitle and the list of sections differ, both driven by the joining metadata. It reuses the look of the existing view and register screens.

Values below are measured from the mock-ups; where the app already has a theme variable or shared component for the same thing, use that instead of the raw value.

Design tokens

Token	Value	Used for
Page background	#f3f3f1	Whole page
Brand green (dark)	#14584a	Headings, links, primary buttons, outline buttons
Top band green	#0e5a49	14 px band at very top
Text	#1c1c1c	Body text
Muted text	#44514c	Hints, counts, captions
Table header bg / text	#e8ece9 / #5b6b66	Column header row, uppercase 14 px, letter-spacing .04em
Row border	#dcdfdc	1 px bottom border per row
Row background	#ffffff	Normal rows
Pending add	bg #dff3e4, text #14582c, row tint #f1faf3	Added rows and chips
Pending remove	bg #fbe3e0, text #8f1f14, row tint #fdf3f2	Removed rows and chips
Warning banner	bg #fff4d6, text #5a4300	Cap reached, load problems
Input border	#8a948f, 40 px high, white bg	All text inputs and selects
Font	Roboto Condensed (existing app font)	Everything. Title 40, section heading 30, row text 17, labels 14

Page structure, top to bottom

1. Top band, 14 px high, brand green.
2. Progress row: a 6 px track with a fill (25% on step 1 of 4) and the text "Step 1 of 4" on the right. Use the existing wizard progress component.
3. Link "‹ Back to search".
4. Page header, a wrapping flex row. Left: title "Update relations: {party name}" (40 px) and a muted subtitle "{legal form or party type label} · {role}", for example "Private liability company (PLC) · Customer" or "Individual · Customer". Right: outline button

"Discard changes" and primary button "Next" (larger, 12 px by 36 px padding). "Next" is disabled while any section is still loading, failed, or violates a rule (for example a PLC with no representative manager), and while a request is in flight.

5. Body: one full-width column with 48 px side padding and a 40 px gap between sections (`min-width: 0`). There is no docked side panel; searching and linking happen in the Link modal described below.
6. Bottom summary bar: white, top border, 14 px by 48 px padding. Left text "N pending changes · X added · Y removed · nothing is saved until you submit". Right: outline "Undo all" and primary "Next". It mirrors the header buttons.

Relation section (one per applicable joining type from the metadata)

Sections stack with a 40 px gap. Each section has:

- **Header row** (wrapping flex, space-between). Left: heading (30 px, brand green), a count pill, and an optional rule chip. Right: a "Find in list" label with a 200 px text input (placeholder "Name or BC number"), and the add button.
- **Count pill**: grey rounded pill showing `effectiveCount/max`, for example `4/99`. Beside it, muted text "3 linked, +1 added, −1 removed" when there are changes.
- **Rule chip** (green) such as "Minimum 1 met"; turns red "Minimum 1 required" when violated. Text comes from the metadata. Shown only for types that have a minimum.
- **Add button**: primary green with a white magnifier icon (16 px) and the joining type label, for example "Ultimate Beneficiary Owner". It opens the Link modal for this section. Disabled with a tooltip when the section is at max.
- **Cap banner** (only at max): warning style, text "Maximum of 99 reached. Remove a party before adding another."
- **Pending additions block** (only when there are added rows): a green label bar "Pending additions (N)" in 14 px uppercase, followed by the added rows.
- **Table**: header row, then rows. Grid columns `minmax(0,2.2fr) minmax(0,1.2fr) minmax(0,1fr) minmax(0,1.1fr) 112px` with 16 px gap and 12 px by 16 px row padding. Columns: Name, BC number, Party ID, Status, action (right aligned text button).
- **Footer row**: left muted text "Showing 1–15 of 52". Right: "Page N of M" with Prev and Next buttons (small outline buttons; disabled ones at 45% opacity). Paging is done on the client over the fully loaded list; page size is 15 and configurable. Hide the footer when the list fits on one page.
- **Names loading hint** (only while filtering by name needs more names): muted text "Loading names… 30 of 52" next to the filter input.

Row states

State	Row background	Text	Status chip	Action
Linked (baseline)	white	normal	grey "Linked"	"Remove"
Pending add	#f1faf3	normal	green "Pending add"	"Discard"
Pending remove	#fdf3f2	struck through, grey #6b6b6b	red "Pending remove"	"Undo"
Loading names	white	skeleton bars for Name and Party ID, BC number shown	none	disabled
Expanded	row as above plus a detail row below			

A row has a small disclosure triangle before the name (▸ collapsed, ▾ expanded). The **detail row** sits directly under its row, background #f8faf9, 14 px by 16 px padding with 40 px left indent, a 3-column grid with small labels (14 px, muted) and values (16 px): Date of birth, Address, BC number.

Link modal (opened from a section's add button)

Based on the existing Link dialog in the register flow (for example "Link Ultimate Beneficiary Owner").

- **Overlay**: the page behind is washed out with a translucent light layer (about rgba(243,243,241,0.7)). The modal is white with a 1 px border #c9cfcb and a soft shadow, centered, about 90% of the viewport width up to 1240 px, with 32 px padding. Its content scrolls inside it; the page behind does not scroll.
- **Header**: title "Link {joining type label}" (28 px) on the left, a close X on the right (40 px click target, aria-label "Close").
- **Tabs** under the header: a text tab with a 2 px brand-green underline on the active one, for example "Individual name". Which tabs appear comes from config and from the allowed participant kinds of the type (natural person gives an individual tab; legal entity gives the organization tab used by the existing search).
- **Form** (existing search component): row one is Initials (160 px), Prefix (130 px), Surname (320 px), Date of birth (200 px, placeholder dd-mm-yyyy, calendar icon). Row two is Country (select, 400 px, default Netherlands (Kingdom of the)), Postcode (160 px, hint "9999 XX" below), House number (140 px). Labels are 14 px above 40 px inputs.

- **Buttons row**: "Register" (dark green filled, left). On the right a text button "Clear Search" and a primary "Search" that is grey and disabled until the existing search rules are satisfied.
- **Results**: below the buttons, a caption "N results" (14 px uppercase muted) and a list of result cards in a 2-column grid. Card: 1 px border #c9cfcb, 12 px padding, name (18 px), second line "BC … · date of birth · postcode" (15 px muted), optional status chip, and one action button on the right. States are in the next section. The list scrolls inside the modal (max height about 40% of the viewport).
- **Footer**: left muted text "N added" (hidden while zero), right primary "Done".

3. FE components, actions and states

Component tree (names are suggestions; match project naming and register everything through the meta-model config, see section 1)

- `RelationsUpdatePage`: header, section list, modal host, summary bar, leave-page guard. Rendered by the shared orchestrator component from a meta-model config entry for this screen. Reads the joining metadata for the party's type and legal form and renders one section per applicable has-a joining type.
- `RelationsUpdateSection` (registered component type): header row, cap banner, pending additions, table, footer. Props: `partyId`, `joiningType`, plus its config block. Gets everything else from the store for that key.
- `RelationTable`, `RelationRow`, `RelationDetailRow`, `PendingAdditions`: presentational parts of the section.
- `LinkPartyModal` (registered component type): the modal that searches and links parties. It wraps the existing party search form and result list. Single instance on the page; the section that opens it sets its target joining type. Its tabs and search fields come from config.
- `SearchResultCard`: one result with its state chip and action, rendered inside the modal.
- `UpdateSummaryBar`: counts and the Undo all / Next buttons.
- Composable `useBatchedPartyDetails`: the micro-batcher described in section 5. A sibling `useBatchedBcResolve` does the same for names.

Link modal behavior

Search and linking happen in a modal, the same pattern as the existing Link dialog in the register flow. The page behind stays where it is, dimmed.

- Opened by the add button of a section. Closed by the X in the top right, by Esc, or by the "Done" button. Focus moves to the first field when it opens, stays trapped inside while open, and returns to the add button that opened it.
- It stays open after an Add, so several parties can be linked in a row. The footer shows "N added" for the current session of the modal; "Done" closes it. Added parties appear in the section's Pending additions block behind the dimmed page.
- Entered search text is kept when the modal is closed and reopened on the same section, and cleared when another section opens it.
- "Register" starts the existing register-party flow for a party that cannot be found. If that flow returns the new party, stage it like a search result (see assumptions).

Search result card states

Every flag below is derived on the client from the store, the metadata and the search result, so no extra server call is needed per candidate.

State	Chip	Button	Rule
Addable	none	"Add"	Passes every check
Already linked	grey "Already a {type} of this party"	"Add" disabled	The section's loaded baseline contains the BC number and it is not marked for removal
In pending additions	green "Added"	"Added" shown as a chip, no button	Draft exists
Removed this session	red "Pending remove"	"Undo removal"	Baseline row currently marked for removal; clicking it undoes the removal instead of staging an add
Maximum reached	yellow "This party already has 99 of 99"	"Add" disabled	`effectiveCount` equals max for the section
Kind not allowed	grey, for example "A UBO must be a natural person"	"Add" disabled	Party kind from the search result is not in the allowed participant kinds from the metadata
Same party	grey "Cannot link a party to itself"	"Add" disabled	Candidate party ID equals the main party ID

State	Chip	Button	Rule
Too many matches	yellow banner instead of results: "More than 10 matches. Add a date of birth or postcode to narrow the search."	none	Search returned more than 10 candidates

User actions

Action	Behavior
Open page	Load metadata. Load every section's full list in parallel (one call per section, see sections 4 and 5). Show skeleton rows for names until the first page of each section is resolved.
Row hover (300 ms debounce) or disclosure click	Request date of birth and address for that party through the micro-batcher. Expanded state is kept per row in the store.
Next / Prev	Client-side paging over the loaded list. Changing page resolves names for the newly visible rows (one batched call of up to 15). No cursors.
Find in list	Filter the section locally. BC number matches immediately across all rows. A text that is not numeric also needs names: resolve the section's remaining unresolved names once in batches of 15 (at most 2 batches at a time), show "Loading names… X of N", and update the filtered rows as batches arrive. Typing again while loading keeps the loaded names and does not restart. Loading is never started without a text filter.
Add button on a section	Open the Link modal for that type.
Search (modal)	Call the existing party search. Cancel any earlier search in flight. Zero results: show "No parties found". More than 10: show the narrow-your-search banner. Otherwise render cards with the states above. Search is disabled until the existing search rules are satisfied.

Action	Behavior
Add (card)	Store `addDraft` with BC number, party ID, name, date of birth, postcode and kind. The card turns to "Added" and the modal's "N added" counter goes up. If the section reaches max, remaining cards turn to "Maximum reached".
Done / X / Esc	Close the modal. Nothing is lost; the drafts live in the store.
Remove (row)	Store `markRemoved`. The row turns pending-remove in place; page boundaries do not change.
Undo / Undo removal / Discard	Reverse the single change in the store.
Undo all	Confirm dialog, then clear every delta in the store.
Next (page or summary bar)	Run client pre-checks (min and max from metadata). If clean, send the delta to the orchestrator endpoint. On success go to the existing Verify screen. On a validation response, mark affected rows or cards with an inline red message and keep the user on the page.
Back to search / Discard changes / browser back	If the delta is non-empty, show the unsaved-changes dialog (Stay or Leave). Same guard for tab close via `beforeunload`.
Keyboard	Enter in the search form runs Search. Esc closes the modal. Tab order inside the modal: tabs, fields, Register, Clear Search, Search, results, Done. Optional: Delete on a focused row toggles remove or undo.

Loading, empty and error states

- Loading names: skeleton bars in Name and Party ID cells. BC number is shown immediately because ZoS already returned it.
- Empty section: a single muted row "No {type} linked yet".
- Section load failure: inline message with a "Retry" text button. Other sections keep working. Next is disabled while any section is failed.

- Section over the cap (more than max rows exist already, a data anomaly): warning banner "This party has more than 99 {type}. It cannot be edited here.", the section is read-only (no add or remove) and Next ignores it.
- Name resolution failure for a row: show the BC number and a small "Details unavailable, retry" link on that row only.
- Modal search failure: inline red message under the form with the error text the project already shows elsewhere.
- Submit failure that is not a validation error: banner above the summary bar with Retry. The delta is kept.

4. FE Pinia store

One store, `useRelationsUpdateStore`, holds the staged edit session for one party. Everything is in memory only: no localStorage, no persistence plugin. The store is reset when the page is left after submit or after the user confirms leaving. The core idea: **the loaded list is an immutable baseline and the user's edits are a delta layered on top.** Because every section holds at most 99 rows, the whole baseline is loaded up front and paged on the client. Edits never move rows between pages.

Config-driven. The store hardcodes no joining types. When `init` runs it takes the sections from the screen's meta-model config entry and the joining metadata for the party, and creates one `SectionState` per section. The registered section and modal components read from this store through the props the orchestrator passes them.

Types (suggested shapes)

```
interface RelationRow {
  joiningId: string
  bcNumber: string
  partyId?: string                    // set after name resolution
  name?: string
  nameState: 'pending' | 'loaded' | 'error'
}

interface AddDraft {                      // a staged ADD
  bcNumber: string
  partyId: string
  name: string
  kind: 'NATURAL_PERSON' | 'LEGAL_ENTITY'
  dateOfBirth?: string
  postcode?: string
}
```

```
interface SectionState {
  joiningType: string
  min: number
  max: number                          // 99
  allowedKinds: string[]               // from metadata
  status: 'idle' | 'loading' | 'ready' | 'error' | 'overCap'
  rows: RelationRow[]                  // the whole baseline, in server order
  pageIndex: number
  pageSize: number                     // 15, configurable
  added: Record<string, AddDraft>      // key = bcNumber
  removed: Record<string, true>        // key = joiningId
  filterText: string
  nameLoad: { running: boolean; loaded: number; total: number }
  expanded: Record<string, boolean>    // key = joiningId
}
```

Session-level maps in the same store: `partyByBc: Record<string, {partyId: string; name: string}>` and `detailsByPartyId: Record<string, {dateOfBirth?: string; address?: string}>`. They exist only to avoid re-requesting within the session.

Link modal state (also in the store): `open`, `joiningType`, `activeTab`, `form` (the search fields), `status`, `results` (candidate cards with their derived state), `tooMany`, `addedCount` (parties added since the modal opened).

Section loading rules

1. `init` loads metadata, then for each applicable joining type calls the section load endpoint once (all sections in parallel). The response holds every baseline row (joiningId and BC number only). Store them in `rows`, set `status = 'ready'`.
2. If the endpoint answers over-cap, set `status = 'overCap'` and make the section read-only.
3. The first page of each section arrives with names already resolved when the endpoint was asked to resolve the first page. Any other visible row without a name is resolved through the batched BC resolve call, 15 at a time.
4. Paging is `pageIndex` over the filtered rows. Prev and Next never call the server for rows; they only trigger name resolution for newly visible rows.
5. Name loading for a text filter: when `filterText` is non-numeric and some rows are unresolved, start one background load (batches of 15, at most 2 in flight), updating `nameLoad`. It never restarts while running and never starts without a filter.

Getters

Getter	Meaning
`filteredRows(type)`	`rows` filtered by `filterText`: BC number always; name only when `nameState = 'loaded'`. Removed rows stay in the list.
`visibleRows(type)`	The current page of `filteredRows`.
`pageCount(type)`	`ceil(filteredRows.length / pageSize)`.
`pendingAdditions(type)`	`Object.values(added)` in insertion order.
`effectiveCount(type)`	`rows.length + addedCount − removedCount`. Always exact.
`isLinked(type, bc)`	`rows` contains the BC number and its joining is not in `removed`.
`isAdded`, `isRemoved`	Lookup in `added` / `removed`.
`atMax(type)`	`effectiveCount >= max`.
`ruleViolations(type)`	Min and max from metadata applied to `effectiveCount`.
`canSubmit`	Every section is `ready` or `overCap`, no rule violations, `hasChanges`, no request in flight.
`hasChanges`	Any section has an `added` or `removed` entry.
`summary`	Counts of added and removed across sections for the bottom bar.
`deltaPayload`	The submit body, see below.

Actions

- `init(partyId)`: load metadata, create one `SectionState` per applicable has-a joining type, load all sections in parallel.
- `loadSection`, `setPage`, `resolveVisible(type)`, `loadNamesForFilter(type)`.
- `setFilter(type, text)`, `toggleExpand(type, joiningId)` (also requests details for that party).
- `openLinkModal(type)`, `closeLinkModal()`, `runSearch()`.
- `addDraft(type, candidate)`: if the candidate's BC number matches a row in `removed`, call `undoRemoved` instead of staging an ADD. If the BC number is already in `added` or already linked, do nothing. If the section is at max, do nothing.

- `discardDraft(type, bc)`, `markRemoved(type, joiningId)`, `undoRemoved(type, joiningId)`, `undoAll()`.
- `submit()`: build `deltaPayload`, call the orchestrator endpoint, handle validation errors, navigate on success.
- `reset()`.

Delta payload

```
{
  "partyId": "33770050",
  "changes": [
    { "op": "ADD",    "joiningType": "UBO", "bcNumber": "2090117745",
"partyId": "33771920" },
    { "op": "REMOVE", "joiningType": "UBO", "bcNumber": "2058870031",
"joiningId": "…" }
  ]
}
```

Only changed joinings are included. Field names follow the existing orchestrator contract where one exists; adapt this shape to it.

Invariants

- A BC number is never in both `added` and the baseline as linked. A baseline row is never in `removed` twice.
- Removing a staged addition discards it; it does not create a REMOVE.
- Removing then re-adding the same baseline party cancels both (no ADD, no REMOVE).
- `effectiveCount` never exceeds `max` through the UI.
- Memory stays small: at most 99 rows per section, each with a few short strings.

5. FE Alova API layer

All calls go through the existing Alova instance (base URL, auth headers and global error handling stay as they are). Store actions own the calls and call `.send()` on method instances; do not tie request lifecycle to component-level `useRequest` hooks. Disable Alova's response cache on these methods (for example `cacheFor: null`, per the Alova version in use): the store's session maps are the only memory, and stale relation data must never be served.

BFF endpoints the FE calls (names are proposals; where the BFF already exposes an equivalent, use it and adapt the types)

Purpose	Request	Response
Joining metadata	existing metadata endpoint, by party type and legal form	has-a joining types with labels, min, max (99), allowed participant kinds, rule texts
Section load	`GET /parties/{partyId}/relations/{joiningType}?resolveFirstPage=true`	`{ items: [{joiningId, bcNumber, partyId?, name?}] }` holding every baseline row, or `{ overCap: true, count }`; with `resolveFirstPage` the first 15 items carry `partyId` and `name`
Resolve names	`POST /parties/resolve` body `{bcNumbers[]}` (up to 15)	`[{bcNumber, partyId, name}]`, with per-item errors
Party details	`POST /parties/details` body `{partyIds[]}`	`[{partyId, dateOfBirth, address}]`, with per-item errors
Party search	existing search API used by the current search screen	candidates with BC number, party ID, name, kind, date of birth, postcode
Submit	existing orchestrator endpoint with the delta payload from section 4	success, or `422` with `violations: [{joiningType, bcNumber?, joiningId?, code, message}]`

There is no candidate check endpoint. Candidate flags are derived on the client (section 3) and fully re-validated by the BFF on submit.

Behaviors to implement in the API layer

- **Micro-batcher for details and names.** A composable `useBatchedPartyDetails` collects `partyId` requests made within a 50 ms window and sends one `POST /parties/details`, chunked to a configurable `MAX_DETAILS_BATCH` (default 20 until

confirmed with the upstream owner). A second instance does the same for BC number resolution, chunked to 15. Each caller gets its own promise.

- **In-flight de-duplication.** A map of `partyId` (or BC number) to pending promise, so hovering and expanding the same row, or two sections asking for the same party, produce one request. Alova shares identical in-flight GETs by default; POSTs need this manual map.
- **Hover and expand.** Row hover waits 300 ms before requesting details; leaving the row before then cancels it. "Expand all on a page" is one batch, not N calls.
- **Name loading for filtering.** Takes the unresolved BC numbers of one section, sends them to the resolve batcher in groups of 15 with at most 2 groups in flight, and reports progress to the store. It can be cancelled when the page is left. Never start it for several sections at once.
- **Section loads.** All sections load in parallel on page open (a party has only a few applicable types). Each is one request; the BFF does the sequential walking.
- **Aborting stale searches.** Keep the last party-search method instance; call its abort before starting a new one, and also guard with a sequence number so a late response from an old search is ignored. Closing the modal aborts the pending search.
- **Errors.** Reuse the global Alova error handling. Map a `422` violations list onto the store: rows by `joiningId`, cards and pending additions by `bcNumber`, otherwise a section banner by `joiningType`. Network errors on a single row's details only mark that row.
- **Timeouts.** Use the app default; the section load may need a longer one because the BFF walks up to 7 ZoS pages.

6. BE functional description (Quarkus BFF)

Implement the endpoints below on the BFF by composing the APIs that already exist (joinings by type, joining metadata, party search by BC number, upstream party details, orchestrator). Only has-a joinings are handled; there is no is-a logic anywhere in these endpoints. The existing all-joinings endpoint used by the view screen stays untouched. Implementation style, resilience annotations, DTO design and tests are left to the usual project practices. The FE contract is in section 5.

Cross-cutting rules

- Every endpoint uses the existing authorization model. The BFF checks that the caller may view or update the party. The FE is never trusted: all rules are re-validated on the BFF.
- Individual and organization parties use the same endpoints. The party type and legal form decide which has-a joining types apply, through the joining metadata.

- No caching and no new persistence. Within one request, identical lookups (same BC number, same party ID) are made once and shared.
- All outbound calls (ZoS, party search, upstream details) run with bounded concurrency, per-call timeouts and an overall request deadline. If the caller disconnects, stop issuing further calls.
- ZoS paging is keyset-based and strictly sequential: the next page is requested with the createdTime and joiningId of the last record. Confirm with the ZoS owner that order is stable (createdTime ascending, joiningId tie-break) so concurrent inserts land at the end of the walk.
- Do not log personal data (date of birth, address). Log correlation ID, party ID, joining type, page counts and latencies.

1. Section load (`GET /parties/{partyId}/relations/{joiningType}?resolveFirstPage=`)

- Check that the joining type is a has-a type applicable to this party's type and legal form per metadata; otherwise return a clear error.
- Walk the ZoS pages for the type sequentially, collecting joiningId and BC number from every row, up to `ceil(max / 15) + 1` pages (8 for max 99). Stop as soon as a page returns fewer than 15 rows, or the collected count exceeds `max`.
- Ended within the cap: return all items in server order. Cap exceeded: stop walking and return an over-cap answer with the count seen so far, and no items.
- With `resolveFirstPage=true`, resolve party ID and name for the first 15 items using the BC party search with bounded concurrency (about 5) and request-scoped dedupe. A row whose resolution fails is returned with its BC number and a `resolveError` flag; the load still succeeds.

2. Resolve names (`POST /parties/resolve`)

- Body is up to 15 BC numbers (reject more). Uses the same resolver as the first-page resolution and returns partial results with per-item errors.

3. Party details (`POST /parties/details`)

- Body is a list of party IDs. Split into chunks of the upstream batch limit (configurable, default 20), call upstream with bounded concurrency, merge. Return date of birth, address and party ID per party. Partial failures are reported per ID without failing the whole call.

4. Submit delta (orchestrator node)

The FE sends only added and removed joinings. Add a node to the existing graph/process orchestrator that:

1. Validates structure: joining types are has-a types applicable to the party, no duplicate ops, no ADD and REMOVE for the same party, the party exists.
2. For each affected joining type, walks the party's current joinings once (same walk as the section load) to get the real baseline: its rows and total.
3. For each REMOVE: confirms the joiningId is in that baseline and its BC number matches the one sent.
4. For each ADD: resolves the candidate, checks the kind against the metadata's allowed participant kinds, rejects a self-link, and rejects a party that is already in the baseline and not being removed.
5. Checks the cap: baseline total plus adds minus removes must not exceed max, per joining type.
6. Applies min rules from metadata to the after-change count, for example a PLC needs at least one representative manager.
7. Collects every violation and answers `422` with a `violations` list (joining type, BC number or joiningId, code, message). Does not stop at the first.
8. On a clean pass, stages the delta into the existing process state so the existing Verify screen can show it and a second person can approve (dual control). For the Verify view, resolve names and details for the changed parties only. After approval, the existing commit step applies all adds and removes as one atomic unit across the ZoS calls (the project's JTA mechanism). Follow the orchestrator's existing structure for where staging and commit sit.
9. Re-submitting the same payload for the same process instance must not create duplicates.

Load protection summary

- Cheap by default: the section load returns BC numbers and joining IDs only, plus names for the first page when asked. Names for other rows come on demand in batches of 15; details only for expanded rows.
- The expensive endpoints (section load, resolve, details) get their own concurrency limits so a burst cannot starve other BFF traffic.
- Hard limits on input sizes (15 BC numbers per resolve call, details batch limit) and on the page walk (cap pages), with clear error codes when exceeded.
- Metrics: ZoS pages per section load and its latency, resolve latency, details latency, upstream error rates.

7. Edge cases, acceptance criteria and assumptions

Edge cases to handle

- A party has exactly 99 rows in a type. The section load must succeed, the add button is disabled and the cap banner shows.
- A party already has more than 99 rows in a type (data anomaly). The load answers over-cap, the section is read-only with a warning, and it is left out of the delta.
- The user removes a baseline party and then adds the same party again. The two cancel; nothing is sent for it.
- The user adds a party that another user has linked in the meantime. The BE returns a violation on that BC number; the pending row shows the message and the user discards it.
- A joining marked for removal was already removed by someone else. The BE reports it by `joiningId`; the row shows the message and the FE drops that op once the user dismisses it.
- A section is still loading or has failed. Next is disabled with a tooltip naming the section.
- Name resolution fails for some rows or the details call fails. Show BC numbers and a per-row retry; do not block the section.
- The user filters by name before all names are loaded. Names are loaded once in batches with a visible progress hint; results update as batches arrive.
- A party search result has no usable party kind. Treat the candidate as not addable with the text "Party kind unknown" until the BE confirms (see assumptions).
- Browser refresh or tab close with staged changes. The leave guard warns; the delta is not restored after a reload.
- A legal form or party type has only one applicable joining type. Show one section.
- Hover storms (mouse dragged down a long table). The 300 ms debounce and the micro-batcher keep this to a few batched calls.

Acceptance criteria

1. A PLC with 2 representative managers and 3 UBOs opens with exact counts (2/99, 3/99) and all rows visible without paging.
2. An individual (minor) opens with the sections the metadata defines for that party type, rendered by the same page.
3. A section with 52 rows shows "Showing 1–15 of 52", pages on the client, and resolves names only for the visible page.
4. Adding a UBO from the Link modal shows it under Pending additions, raises the counter, enables Next, and sends a payload with exactly one ADD.
5. Marking a row for removal strikes it through in place, lowers the counter, and does not change page boundaries. Undo restores it and the payload is empty again.
6. At 99 of 99, every Add is disabled, the cap banner shows, and removing one row re-enables adding.

7. Filtering by BC number is instant. Filtering by name loads the remaining names once, with progress, then filters all rows.
8. Opening the page triggers one section load per applicable type. Expanding all rows on a page triggers one details call.
9. Submitting with a rule violation (for example removing the only representative manager of a PLC) shows the BE message on the section and does not leave the page.
10. A successful submit lands on the existing Verify screen, which lists only the changed relations.
11. Leaving with staged changes asks for confirmation; leaving after a successful submit does not.
12. No endpoint accepts more than its input limit, and none of them logs date of birth or address.

Assumptions to confirm before building

Assumption	Why it matters	If wrong
BC party search returns party ID and name	Names are resolved this way for every visible page	Names must come from the details API, which makes expanding part of the first page load
Party search results include party kind (natural person or legal entity)	Candidate kind check is done on the client	Add a small BE endpoint that validates candidates by BC number, or check only on submit
ZoS orders by createdTime ascending with joiningId tie-break and ends with a page under 15 rows	Stable walk and end detection	Adjust the walk and the end test
Upstream details batch limit is about 20	Chunk size	Change `MAX_DETAILS_BATCH`
Loading all rows of a type (up to 8 sequential ZoS calls) on page open is acceptable	Replaces cursor paging in the UI	Fall back to loading page by page and paging with the cursor, at the cost of an exact count
Existing orchestrator contract has its own field names for the delta	Payload shape in section 4 is a proposal	Map to the existing contract

Assumption	Why it matters	If wrong
Verify screen can render a delta of changed relations only	Avoids fetching unchanged parties	Fetch the minimum extra data it needs
The existing meta-model config and shared orchestrator can declare these two component types and the screen's sections	The screen must be config-driven like the others	Extend the config schema or add a thin adapter, but keep sections and modal tabs in config
The existing register flow can hand the newly registered party back to the Link modal	The Register button lives inside the modal	The modal closes after registering and the user searches for the new party